Projekt: Software Engineering

(DLMCSPSE01_D)

IU Internationale Hochschule

Studiengang: Informatik

Architekturdokument

Jessica Krautheuser

Matrikelnummer: IU14129800

Tutor: Prof. Dr. Dirk Simon

Abgabedatum: 30.10.2026

Inhalt

Abbildungsverzeichnis III

Abkürzungsverzeichnis IV

1. Einleitung 1

2. Technologieübersicht 1

2.1 Python 1

2.2 HTML, CSS und JavaScript 2

2.3 FastAPI, Uvicorn und Client-Server-Kommunikation 2

2.4 pytest 3

2.5 Entwicklungs- und Dokumentationswerkzeuge 4

3. Architekturübersicht 5

4. Struktur 7

5. Verhalten 11

Literaturverzeichnis IV

Abbildungsverzeichnis

Abbildung 1: Architekturdiagramm 6
Abbildung 2: UML-Klassendiagramm 10
Abbildung 3: Sequenzdiagramm Spielzug 12

Abkürzungsverzeichnis

ASGI	Asynchronous Server Gateway Interface
CSS	Cascading Style Sheets
GUI	Graphic User Interface
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrierte Entwicklungsumgebung
JSON	JavaScript Object Notation
PyPI	Python Package Index
TLS	Transport Layer Security
UML	Unified Modeling Language

1. Einleitung

Dieses Dokument beschreibt die Softwarearchitektur der Webanwendung Mystic Maze. Ziel ist es, die technischen Grundlagen für die Implementierung festzulegen und die interne Struktur der Anwendung nachvollziehbar zu dokumentieren. Dazu werden die eingesetzten Technologien und Werkzeuge, die gewählte Architektur sowie die zentralen Komponenten und ihre Beziehungen beschrieben. Ergänzend werden die Struktur der Anwendung durch ein UML-Klassendiagramm und ein zentraler Systemablauf durch ein UML-Sequenzdiagramm dargestellt. Das Architekturdokument baut auf den zuvor erstellten Anforderungen und der Spezifikation auf und dient als technische Grundlage für die weitere Implementierung der Anwendung.

2. Technologieübersicht

Für die Umsetzung von Mystic Maze werden Technologien eingesetzt, die eine browserbasierte Mehrspieler-Anwendung mit zentral verwalteter Spiellogik und synchronisierten Spielzuständen ermöglichen. Bei der Auswahl wird insbesondere auf eine klare Trennung der Spiellogik von der Benutzeroberfläche, eine einfache Client-Server-Kommunikation sowie eine gute Testbarkeit geachtet.

2.1 Python

Die serverseitige Anwendung und die Spiellogik werden in Python implementiert. Für den Einsatz der Programmiersprache spricht das bereits vorhandene Know-how bei der Entwicklerin. Dadurch ist nur eine geringe Einarbeitung notwendig und der Fokus kann auf der Umsetzung der Spiellogik liegen.

Python besitzt eine übersichtliche Syntax und unterstützt objektorientierte Programmierung. Dadurch lassen sich Bestandteile des Spiels wie Spielfeld, Kacheln, Spieler und Partien durch einzelne Klassen abbilden. Die offizielle Python-Dokumentation beschreibt Python als leicht zu erlernende Programmiersprache mit einer klaren Syntax sowie einer umfangreichen Standardbibliothek (Python Software Foundation, o. J.-b, o. S.).

Ein weiterer Vorteil ist die umfangreiche Auswahl von Bibliotheken, Modulen und Frameworks. Über den Python Package Index (PyPI) steht eine große Anzahl zusätzlicher Softwarepakete zur Verfügung, sodass benötigte Funktionen bei Bedarf durch etablierte Bibliotheken ergänzt werden können (Python Software Foundation, o. J.-a, o. S.). Für Mystic Maze kann Python deshalb sowohl für die Spiellogik als auch für die serverseitige Bereitstellung der Anwendung verwendet werden.

2.2 HTML, CSS und JavaScript

Die grafische Benutzeroberfläche und das Frontend von Mystic Maze wird mit HTML (HyperText Markup Language), CSS (Cascading Style Sheets) und JavaScript umgesetzt. HTML bildet die Struktur der einzelnen Webseiten und deren Inhalte. CSS wird für die Gestaltung und Anordnung der Oberflächenelemente verwendet. JavaScript übernimmt die Interaktivität und das dynamische Verhalten der Anwendung (Mozilla, o. J., o. S.).

Bei Mystic Maze verarbeitet JavaScript die Eingaben der Spieler und übernimmt die Kommunikation mit dem Server. Dazu gehören bspw. das Drehen und Einschieben einer Kachel, die Auswahl eines Zielfeldes und das Beenden eines Spielzugs. Außerdem werden die vom Server empfangenen Änderungen des Spielzustands durch JavaScript in der Benutzeroberfläche dargestellt.

Für die Kombination aus HTML, CSS und JavaScript spricht vor allem, dass alle drei Technologien direkt von modernen Webbrowsern unterstützt werden. Die Spieler benötigen deshalb keine zusätzliche Software, sondern können das Spiel über ihren Browser aufrufen. Außerdem lassen sich die Struktur, Gestaltung und Funktionen der Benutzeroberfläche getrennt voneinander entwickeln.

Auf ein zusätzliches Frontend-Framework wird verzichtet, da sich der benötigte Funktionsumfang mit diesen drei Technologien umsetzen lässt. Dadurch werden zusätzliche Abhängigkeiten und der damit verbundene Entwicklungsaufwand vermieden. Die eigentliche Spiellogik bleibt weiterhin auf dem Python-Server und wird nicht im Browser ausgeführt.

2.3 FastAPI, Uvicorn und Client-Server-Kommunikation

Als serverseitiges Webframework wird FastAPI eingesetzt. FastAPI ist ein Python-Framework zur Entwicklung von Web-APIs und basiert auf standardmäßigen Python-Typannotationen (FastAPI, o. J.-a, o. S.). Für die Definition und Validierung strukturierter Ein- und Ausgabedaten wird ergänzend Pydantic verwendet. Die Datenmodelle werden dabei mithilfe von Python-Typannotationen beschrieben und von FastAPI direkt für die Verarbeitung von Anfragen und Antworten genutzt.

Für einzelne Anfragen zwischen Browser und Server wird das Hypertext Transfer Protocol (HTTP) verwendet. HTTP ist ein zustandsloses Protokoll auf Anwendungsebene und basiert auf einem Request-Response-Prinzip zwischen Client und Server (Fielding et al., 2022, S. 19). Bei Mystic Maze wird HTTP beispielsweise für das Aufrufen der Webanwendung sowie für das Erstellen, Beitreten und Starten einer Partie verwendet. Statt HTTP wäre es auch möglich HTTPS zu verwenden. HTTPS ermöglicht durch den Einsatz von TLS (Transport Layer Security) eine verschlüsselte Kommunikation zwischen Webbrowser und Webserver (Fielding et al., 2022, S. 25). Da Mystic Maze ausschließlich für die Nutzung innerhalb eines lokalen WLANs vorgesehen ist, wird im Rahmen des Projektumfangs auf die zusätzliche Einrichtung und Verwaltung von TLS-Zertifikaten verzichtet. Die Datenübertragung erfolgt daher unverschlüsselt.

Während der Spielvorbereitung und einer laufenden Partie müssen Änderungen des Spielzustands auch ohne eine neue HTTP-Anfrage an die verbundenen Spieler übertragen werden können. Dafür wird das WebSocket-Protokoll verwendet. WebSocket ermöglicht nach dem Verbindungsaufbau eine bidirektionale Kommunikation zwischen Client und Server. Beide Seiten können dabei unabhängig voneinander Nachrichten übertragen (Fette & Melnikov, 2011, S. 5). Bei Mystic Maze werden über diese Verbindung Spielaktionen an den Server gesendet und Änderungen des Spielzustands an die beteiligten Clients verteilt.

Für die Übertragung strukturierter Daten zwischen Browser und Server wird JSON (JavaScript Object Notation) verwendet. JSON ist ein textbasiertes und sprachunabhängiges Format zur Darstellung strukturierter Daten (Bray, 2017, S. 1) . Dadurch können beispielsweise Partie-IDs, Spielerinformationen, Spielaktionen, Positionen und weitere Bestandteile des Spielzustands in einer einheitlichen Form zwischen dem JavaScript-Frontend und dem Python-Backend ausgetauscht werden.

Für Mystic Maze wurde FastAPI als Framework gewählt, da es sowohl HTTP-Endpunkte als auch WebSocket-Verbindungen unterstützt. Dadurch können beide Kommunikationsarten mit einem Framework umgesetzt werden. FastAPI stellt außerdem Funktionen zum Senden und Empfangen von Text-, Binär- und JSON-Daten über WebSocket-Verbindungen bereit (FastAPI, o. J.-c, o. S.).

Zur Ausführung der FastAPI-Anwendung wird Uvicorn verwendet. FastAPI basiert auf dem Asynchronous Server Gateway Interface (ASGI) und benötigt für die Bereitstellung der Anwendung einen entsprechenden ASGI-Server. Uvicorn übernimmt diese Aufgabe und verarbeitet die eingehenden HTTP- und WebSocket-Verbindungen (FastAPI, o. J.-b, o. S.). Dadurch kann die serverseitige Anwendung von Mystic Maze im lokalen Netzwerk bereitgestellt werden.

2.4 pytest

Für die automatisierten Tests von Mystic Maze wird das Python-Testframework pytest verwendet. Mit pytest lassen sich Tests erstellen und automatisch ausführen. Die Testbedingungen können dabei mit den normalen assert-Anweisungen von Python überprüft werden. Schlägt ein Test fehl, zeigt pytest zusätzliche Informationen zu den beteiligten Werten an und erleichtert dadurch die Fehlersuche. Außerdem erkennt pytest Testdateien und Testfunktionen anhand festgelegter Namenskonventionen automatisch (pytest.org, o. J., o. S.).

Bei Mystic Maze wird pytest vor allem zur Überprüfung der Spiellogik eingesetzt. Dazu gehören beispielsweise die Erstellung und Verschiebung des Spielfelds, die Ermittlung erreichbarer Positionen und die Einhaltung der Spielregeln. Da die Spiellogik von der Benutzeroberfläche getrennt ist, können diese Funktionen unabhängig vom Browser getestet werden. So lassen sich Fehler in den Spielmechaniken bereits während der Entwicklung erkennen.

Für die Wahl von pytest spricht außerdem, dass die Anwendung bereits in Python entwickelt wird. Die Tests können deshalb in derselben Programmiersprache geschrieben werden und benötigen

kein zusätzliches Testframework einer anderen Sprache. Bei Änderungen an der Spiellogik können die Tests erneut ausgeführt und auch für die abschließende Qualitätssicherung verwendet werden.

2.5 Entwicklungs- und Dokumentationswerkzeuge

Für die Entwicklung von Mystic Maze wird PyCharm als integrierte Entwicklungsumgebung (IDE) verwendet. PyCharm bietet Funktionen zur Bearbeitung und Verwaltung des Python-Codes sowie zur Ausführung und Fehlersuche während der Entwicklung (JetBrains, o. J.-b, o. S.). Für die Wahl von PyCharm spricht vor allem die Unterstützung der Programmiersprache Python und die Möglichkeit, den Quellcode, die Tests und die Ausführung der Anwendung innerhalb einer Entwicklungsumgebung zu verwalten.

Zusätzlich wird in PyCharm das Plugin AI Assistant mit dem KI-Agenten Codex von OpenAI verwendet. Codex kann auf den Projektcode zugreifen und bei der Entwicklung neuer Funktionen, der Fehlersuche und der Bearbeitung bestehender Dateien unterstützen (JetBrains, o. J.-a, o. S.). Bei Mystic Maze wird Codex für die Fehlerbehebung und die Gestaltung der Benutzeroberfläche eingesetzt.

Für die Erstellung der Grafiken von Mystic Maze wird der Chatbot ChatGPT von OpenAI verwendet. Dazu gehören beispielsweise die Labyrinthkacheln, der Waldbodenhintergrund und verschiedene Symbole für die Benutzeroberfläche. Die Grafiken werden mithilfe der integrierten Bildgenerierung durch Textanweisungen erstellt und anschließend durch weitere Anweisungen angepasst (OpenAI, o. J., o. S.).

Zur Versionsverwaltung wird Git eingesetzt. Git ist ein verteiltes Versionsverwaltungssystem, mit dem unterschiedliche Entwicklungsstände und Änderungen am Quellcode nachvollzogen werden können (Git, o. J., o. S.). Das zugehörige Repository wird auf GitHub bereitgestellt. Ein GitHub-Repository kann Quellcode und weitere Projektdateien einschließlich ihrer Versionshistorie zentral verwalten (GitHub, o. J., o. S.).

Zur Modellierung von UML-Diagrammen wird das Tool PlantUML eingesetzt. Mit PlantUML können Diagramme durch einen textbasierten Code beschrieben und daraus automatisch grafische Darstellungen erzeugt werden (PlantUML, o. J., o. S.). Dadurch lassen sich die Diagramme bei Änderungen an der Architektur direkt im Code anpassen, ohne die einzelnen Elemente manuell verschieben oder neu zeichnen zu müssen.

Die Mockups der Benutzeroberfläche im Spezifikationsdokument wurden mithilfe von Balsamiq erstellt. Balsamiq ist ein Werkzeug zur Erstellung von Mockups, mit denen die grundlegende Struktur und Anordnung von Elementen einer GUI bereits vor deren Implementierung visualisiert werden kann (Balsamiq, o. J., o. S.).

3. Architekturübersicht

Mystic Maze wird als webbasierte Client-Server-Anwendung umgesetzt. Bei einer Client-Server-Architektur fordert ein Client Dienste beziehungsweise Daten von einem Server an, während der Server diese bereitstellt und die entsprechenden Anfragen verarbeitet. Webanwendungen stellen einen typischen Anwendungsfall dieser Architektur dar (Salah et al., 2016, S. 319).

Bei Mystic Maze übernehmen die Webbrowser der Spieler die Rolle der Clients. Sie stellen die GUI dar, erfassen Benutzeraktionen und übertragen diese an den Server. Die zentrale Spiellogik und der verbindliche Spielzustand werden dagegen serverseitig verwaltet. Dadurch existiert für jede Partie ein eindeutiger Zustand, der nicht unabhängig auf den einzelnen Endgeräten berechnet werden muss.

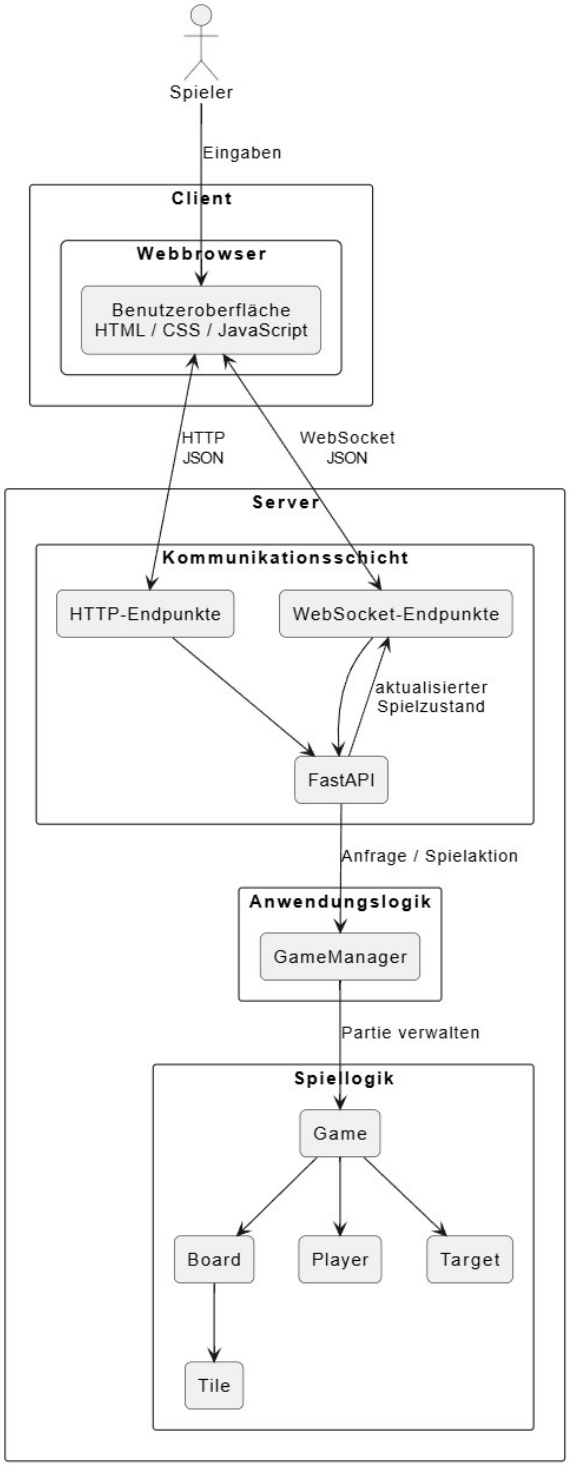
Bei Übertragung der Spielzustände wird zwischen öffentlichen und privaten Informationen unterschieden. Öffentliche Informationen sind Informationen, die für alle Spieler relevant sind, dazu zählen u.a. das Spielfeld, die Positionen der Spieler und der Zugstatus. Persönliche Informationen werden nur für einen Spieler gebraucht, bspw. das aktuell gesuchte Ziel des Spielers, und werden auch nur an den Client dieses Spielers gesendet.

Diese zentrale Verwaltung ist insbesondere für den Mehrspielermodus relevant. Aktionen eines Spielers, beispielsweise das Drehen oder Einschieben einer Kachel oder die Bewegung einer Spielfigur, werden zunächst an den Server übertragen. Dort wird geprüft, ob die Aktion im aktuellen Spielzustand zulässig ist. Nur gültige Aktionen verändern den zentralen Spielzustand. Anschließend wird der aktualisierte Zustand an die verbundenen Clients verteilt. Dadurch soll verhindert werden, dass unterschiedliche Endgeräte voneinander abweichende Spielzustände verwenden.

Innerhalb der serverseitigen Anwendung werden die Verantwortlichkeiten zusätzlich in voneinander abgegrenzte Bereiche unterteilt. Dieses Vorgehen orientiert sich am Prinzip einer geschichteten Architektur. In einem solchen System sind die einzelnen Schichten hierarchisch angeordnet und stellen jeweils Dienste für andere Schichten bereit. Durch die Aufteilung in einzelne Schichten können Änderungen an einem Bereich vorgenommen werden, ohne dass andere Bereiche direkt davon betroffen sind (Garlan & Shaw, 1993, S. 9-11).

Für Mystic Maze wird zwischen Benutzeroberfläche, Kommunikationsschicht, Anwendungslogik und Spiellogik unterschieden. Die daraus resultierende grundlegende Architektur ist in der folgenden Abbildung dargestellt. Die Abbildung zeigt die Aufteilung der Anwendung in den browserbasierten Client und die serverseitigen Bereiche sowie die wesentlichen Kommunikations- und Abhängigkeitsbeziehungen zwischen diesen Komponenten.

Abbildung 1: Architekturdiagramm



Quelle: Eigene Darstellung

Die Benutzeroberfläche wird im Browser ausgeführt und stellt den Spielzustand dar sowie die verfügbaren Benutzeraktionen bereit. Die Kommunikationsschicht bildet mit FastAPI die HTTP- und WebSocket-Schnittstellen ab und nimmt Nachrichten der Clients entgegen. HTTP wird für abgegrenzte Vorgänge wie das Erstellen, Beitreten und Starten einer Partie sowie für den initialen Abruf des Spielzustands verwendet. Für die fortlaufende Kommunikation in der Lobby und während einer

laufenden Partie wird anschließend eine WebSocket-Verbindung aufgebaut. Über diese werden Spielaktionen an den Server übertragen und Änderungen des Spielzustands an die verbundenen Clients verteilt. Die übertragenen Daten werden jeweils als JSON strukturiert.

Die Anwendungslogik koordiniert die vorhandenen Partien und vermittelt zwischen der Kommunikation mit den Clients und der eigentlichen Spiellogik. Die Spiellogik enthält die zentralen Geschäftsobjekte und Spielregeln und bleibt weitgehend unabhängig von der grafischen Benutzeroberfläche und der Netzwerkkommunikation. Eine zentrale Rolle innerhalb der Anwendungslogik übernimmt der GameManager. Er verwaltet die vorhandenen Partien und stellt die Verbindung zwischen der Kommunikationsschicht und den einzelnen Spielinstanzen her. Eine konkrete Partie wird durch die Klasse Game repräsentiert, die unter anderem Spieler, Spielzustand und Zugsteuerung verwaltet. Weitere Klassen wie Board und Target bilden die eigentlichen Bestandteile und Regeln des Spiels ab.

Auf eine Microservice-Architektur oder eine Aufteilung auf mehrere unabhängige Serverdienste wird bewusst verzichtet. Microservice-Architekturen können Vorteile hinsichtlich Skalierbarkeit, Resilienz und unabhängiger Bereitstellung einzelner Dienste bieten. Allerdings gehen sie mit zusätzlichem Aufwand für Kommunikation, Koordination, Sicherheit und Überwachung der einzelnen Dienste einher (Salah et al., 2016, S. 323–324). Da Mystic Maze nur einen begrenzten Funktionsumfang besitzt und hauptsächlich innerhalb eines lokalen Netzwerks verwendet werden soll, werden diese Vorteile nicht benötigt. Eine Verteilung auf mehrere Serverdienste würde die Entwicklung und Verwaltung der Anwendung komplizierter machen, ohne einen entsprechenden Nutzen für das Projekt zu bieten.

Die gewählte Architektur unterstützt gleichzeitig die Wartbarkeit und Testbarkeit der Anwendung. Da die Spiellogik von der Benutzeroberfläche und der Netzwerkkommunikation getrennt wird, können zentrale Spielregeln unabhängig vom Webbrowser getestet werden. Änderungen an der grafischen Darstellung sollen dadurch ebenfalls möglich sein, ohne die grundlegende Spiellogik verändern zu müssen.

4. Struktur

Die interne Struktur von Mystic Maze baut auf dem Datenmodell aus der Spezifikation auf. Dort werden die fachlichen Zusammenhänge zwischen Partie, Spieler, Spielfeld, Kachel und Ziel beschrieben. Im folgenden Kapitel werden die dazugehörigen Python-Klassen und deren Aufgaben genauer erläutert.

Die Verwaltung mehrerer laufender Partien übernimmt der GameManager. Er hält die vorhandenen Game-Instanzen vor und stellt Operationen zum Erstellen, Abrufen und Entfernen einer Partie sowie zum Hinzufügen und Abrufen von Spielern bereit. Jede Partie erhält eine eindeutige Partie-ID, über die eingehende Anfragen der richtigen Game-Instanz zugeordnet werden können.

Beim Beitritt erhält jeder Spieler zusätzlich einen festen Index innerhalb der Spielerliste. Dieser wird als player_index bezeichnet und dient zur Identifizierung des Spielers innerhalb einer Partie. Zusammen mit der Partie-ID kann die Kommunikationsschicht dadurch Anfragen und WebSocket-Verbindungen dem jeweiligen Spieler zuordnen.

Die Klasse Game bildet eine einzelne Partie ab und enthält den wesentlichen Spielzustand sowie die Steuerung des Spielablaufs. Dazu gehören unter anderem die teilnehmenden Spieler, das Spielfeld, der aktive Spieler, der aktuelle Zugstatus, bereits aktivierte Schlüssel sowie der Status der Partie. Game koordiniert außerdem zentrale Spielaktionen wie das Starten einer Partie, das Einschieben der freien Kachel, das Bewegen eines Spielers, die Verarbeitung von Spezialeffekten und das Beenden eines Spielzugs. Bereits in der bestehenden Spiellogik werden beispielsweise der Verschiebe- und Bewegungsstatus sowie der Gewinner innerhalb der Game-Instanz verwaltet.

Das eigentliche Spielfeld wird durch die Klasse Board repräsentiert. Sie verwaltet das 7 × 7-Kachelraster sowie die aktuell freie Kachel. Zu ihren Aufgaben gehören insbesondere die Manipulation des Spielfelds durch das Einschieben der freien Kachel und die Ermittlung erreichbarer Positionen. Die Entscheidung, ob eine konkrete Spielaktion gemäß dem aktuellen Zug erlaubt ist, verbleibt dagegen überwiegend bei Game. Dadurch wird die geometrische Spielfeldlogik von der übergeordneten Regel- und Zugsteuerung getrennt.

Eine einzelne Labyrinthkachel wird durch Tile dargestellt. Eine Kachel besitzt insbesondere einen Kacheltyp und eine Orientierung. Zusätzlich kann sie ein Ziel, ein Tor oder ein Portal enthalten. Die Ausprägungen von Kacheltypen, Richtungen und Toren werden durch Enums wie TileType, Direction und GateType abgebildet. Dadurch werden nur vordefinierte und fachlich zulässige Werte innerhalb der Spiellogik verwendet.

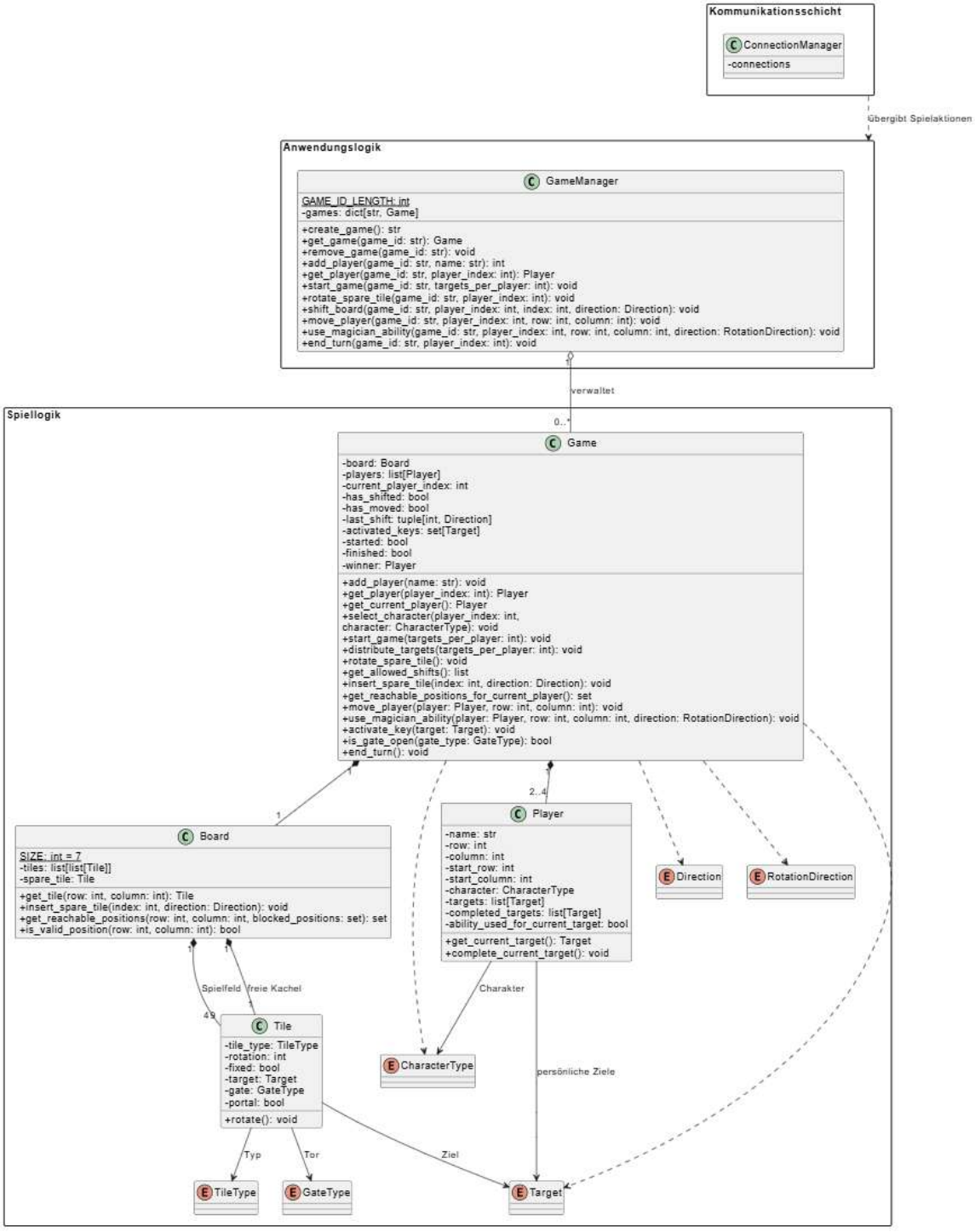
Die Klasse Player speichert die Informationen eines einzelnen Spielers. Dazu gehören der Name, die aktuelle Position, die Startposition, der gewählte Charakter sowie die noch offenen und bereits abgeschlossenen Ziele. Außerdem wird gespeichert, ob die Charakterfähigkeit für das aktuelle Ziel bereits verwendet wurde. Über die Methode get_current_target() kann das aktuell gesuchte Ziel abgerufen werden. Diese Information benötigt Game beispielsweise, um zu überprüfen, ob ein Spieler sein Ziel erreicht hat.

Die möglichen Ziele und Artefakte werden durch die Enumeration Target festgelegt. Anders als im fachlichen Datenmodell der Spezifikation werden Zielkarten und Artefakte nicht durch getrennte Klassen dargestellt. Stattdessen werden die Werte von Target sowohl den Spielern als persönliche Ziele als auch den Kacheln als auffindbare Objekte zugeordnet. Besondere Ziele wie die Schlüssel werden zusätzlich von Game verarbeitet. Wird ein Schlüssel aktiviert, verändert sich der Zustand des dazugehörigen Tores.

Für die Verwaltung der WebSocket-Verbindungen wird die Klasse ConnectionManager verwendet. Sie gehört zur Kommunikationsschicht und verwaltet die aktiven Verbindungen der einzelnen Spieler. Eingehende Spielaktionen werden von der Kommunikationsschicht an den GameManager weitergegeben. Dieser ordnet die Aktionen der entsprechenden Game-Instanz zu, in der die Spielregeln überprüft und die Aktionen verarbeitet werden. Anschließend wird der aktualisierte Spielzustand über die Kommunikationsschicht und den ConnectionManager an die verbundenen Clients übertragen. Dadurch bleiben die Verwaltung der Verbindungen und die eigentliche Spiellogik voneinander getrennt. Ergänzend zum ConnectionManager werden Hilfsmodule eingesetzt. Diese übernehmen unter anderem die Bereitstellung der HTTP- und WebSocket-Schnittstellen, die Verarbeitung eingehender Spielaktionen sowie die Verteilung aktualisierter Spielzustände an die verbundenen Clients.

Die Klassen sind im folgenden UML-Klassendiagramm dargestellt. Aus Gründen der Übersichtlichkeit werden dort ausschließlich die wesentlichen Methoden, die zur Spiellogik beitragen, aufgeführt. Hilfsmodule und interne Hilfsfunktionen werden nicht dargestellt. Die Hilfsmodule dienen der Trennung technischer Aufgaben, während die internen Hilfsfunktionen einzelne Verarbeitungsschritte strukturieren. Beide verändern die grundlegenden Beziehungen zwischen den dargestellten Klassen nicht.

Abbildung 2: UML-Klassendiagramm



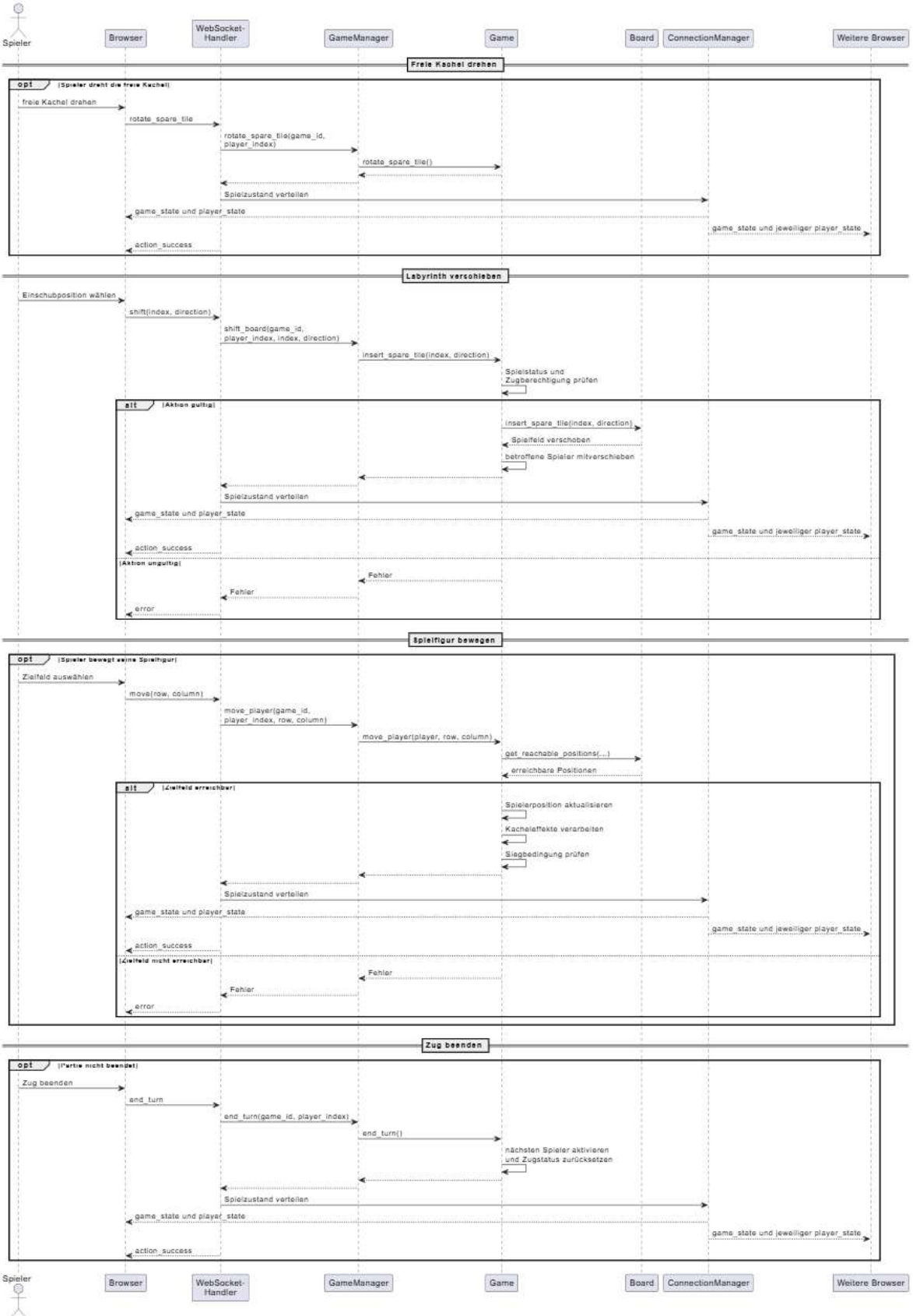
Quelle: Eigene Darstellung

Die dargestellte Struktur folgt der in Kapitel 3 beschriebenen Trennung der Verantwortlichkeiten. GameManager und ConnectionManager übernehmen übergeordnete Verwaltungs- beziehungsweise Kommunikationsaufgaben, während Game, Board, Tile und Player die eigentliche Spiellogik und den Spielzustand abbilden. Dadurch bleibt insbesondere die Spiellogik unabhängig von der konkreten Weboberfläche und kann separat automatisiert getestet werden.

5. Verhalten

Zur Visualisierung eines zentralen Systemablaufs wird der Ablauf eines Spielzugs als UML-Sequenzdiagramm dargestellt. Optionale Sonderaktionen wie die Verwendung einer Charakterfähigkeit werden aus Gründen der Übersichtlichkeit nicht einzeln dargestellt. Auch diese Aktionen werden an den Server übertragen, dort überprüft und verarbeitet. Anschließend wird der aktualisierte Spielzustand an die verbundenen Spieler gesendet.

Abbildung 3: Sequenzdiagramm Spielzug



Quelle: Eigene Darstellung

Das UML-Sequenzdiagramm zeigt den Ablauf eines Spielzugs von Mystic Maze und die Kommunikation zwischen den beteiligten Komponenten. Die Spielaktionen werden vom Browser über die WebSocket-Verbindung an den Server übertragen. Dort werden sie durch den GameManager der entsprechenden Game-Instanz zugeordnet. Game überprüft die Spielregeln und führt die Aktionen aus. Für das Verschieben des Spielfelds und die Ermittlung erreichbarer Positionen wird die Klasse Board verwendet.

Während eines Spielzugs kann der Spieler zunächst die freie Kachel drehen und muss anschließend das Spielfeld verschieben. Danach kann er seine Spielfigur bewegen oder den Zug direkt beenden. Nach jeder erfolgreichen Spielaktion wird der aktualisierte Spielzustand an die verbundenen Clients übertragen. Die Verteilung wird durch die Kommunikationsschicht koordiniert, wobei der ConnectionManager die aktiven WebSocket-Verbindungen verwaltet. Ungültige Aktionen werden nicht ausgeführt und der Spieler erhält eine Fehlermeldung.

Literaturverzeichnis

Balsamiq. (o. J.). *What is Balsamiq? Low-fidelity wireframing tool - Balsamiq Help Center*. Abgerufen 23. Oktober 2026, von https://balsamiq.com/support/getting-started/what-is-balsamiq/

Bray, T. (2017). The JavaScript Object Notation (JSON) Data Interchange Format. *Internet Engineering Task Force (IETF)*, 1–15. https://doi.org/10.17487/RFC8259

FastAPI. (o. J.-a). *Features - FastAPI*. Abgerufen 23. Oktober 2026, von https://fastapi.tiangolo.com/features/

FastAPI. (o. J.-b). *Run a Server Manually - FastAPI*. Abgerufen 23. Oktober 2026, von https://fastapi.tiangolo.com/deployment/manually/#use-the-fastapi-run-command

FastAPI. (o. J.-c). *WebSockets - FastAPI*. Abgerufen 23. Oktober 2026, von https://fastapi.tiangolo.com/advanced/websockets/

Fette, I., & Melnikov, A. (2011). The WebSocket Protocol. *Internet Engineering Task Force (IETF)*, 1–69. https://doi.org/10.17487/RFC6455

Fielding, R., Nottingham, M., & Reschke, J. (2022). HTTP Semantics. *Internet Engineering Task Force (IETF)*, 1–194. https://doi.org/10.17487/RFC9110

Garlan, D., & Shaw, M. (1993). *AN INTRODUCTION TO SOFTWARE ARCHITECTURE* (S. 1–39). https://doi.org/10.1142/9789812798039_0001

Git. (o. J.). *Git - user-manual Documentation*. Abgerufen 23. Oktober 2026, von https://git-scm.com/docs/user-manual

GitHub. (o. J.). *About repositories - GitHub Docs*. Abgerufen 23. Oktober 2026, von https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories

JetBrains. (o. J.-a). *AI Assistant in JetBrains IDEs | PyCharm Documentation*. Abgerufen 26. Oktober 2026, von https://www.jetbrains.com/help/pycharm/ai-assistant-in-jetbrains-ides.html

JetBrains. (o. J.-b). *Quick start guide | PyCharm Documentation*. Abgerufen 23. Oktober 2026, von https://www.jetbrains.com/help/pycharm/quick-start-guide.html

Mozilla. (o. J.). *Your first website - Learn web development | MDN*. Abgerufen 23. Oktober 2026, von https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Your_first_website

OpenAI. (o. J.). *Images in ChatGPT*. Abgerufen 29. Oktober 2026, von https://help.openai.com/en/articles/11084440-images-in-chatgpt

PlantUML. (o. J.). *Open-Source-Tool, das einfache Textbeschreibungen verwendet UML-Diagramme zu zeichnen.* Abgerufen 23. Oktober 2026, von https://plantuml.com/de/

pytest.org. (o. J.). *pytest documentation*. Abgerufen 23. Oktober 2026, von https://docs.pytest.org/en/latest/

Python Software Foundation. (o. J.-a). *PyPI Docs*. Abgerufen 23. Oktober 2026, von https://docs.pypi.org/

Python Software Foundation. (o. J.-b). *The Python Tutorial — Python 3.15.1 documentation*. Abgerufen 23. Oktober 2026, von https://docs.python.org/3/tutorial/

Salah, T., Jamal Zemerly, M., Chan Yeob Yeun, Al-Qutayri, M., & Al-Hammadi, Y. (2016). The evolution of distributed systems towards microservices architecture. *2016 11th International Conference for Internet Technology and Secured Transactions (ICITST)*, 318–325. https://doi.org/10.1109/ICITST.2016.7856721